

Task 04 - Part 01 & Part 02: Question Answers

C# Fundamentals - Arrays, Loops, Switch Statements & Enums

Q1. What is the default value assigned to array elements in C#?

When an array is created without explicit initialization, each element is automatically set to the **default value of its element type**. For numeric types (int, double, etc.) the default is **0**, for bool it's **false**, for char it's the null character ('\0'), and for any reference type (such as string or a custom class) the default is **null**. This happens because the .NET runtime zero-initializes the memory allocated for the array.

Q2. What is the difference between Array.Clone() and Array.Copy()?

Array.Clone() is an instance method that creates a completely new array object containing a **shallow copy** of the original - for arrays of value types, this behaves like an independent copy; for arrays of reference types, only the references are copied, not the objects they point to. It always produces a full copy of the whole array.

Array.Copy() is a static method that copies a specified number of elements from an existing source array into an existing destination array (which must already be allocated). It gives more control - you can copy only a portion of the array, or copy between two different existing arrays - whereas Clone() always duplicates the entire array into a brand-new one.

Q3. What is the difference between GetLength() and Length for multidimensional arrays?

Length is a property that returns the **total number of elements across all dimensions** of the array (e.g., for a 3x3 2D array, Length returns 9). **GetLength(dimension)** is a method that returns the number of elements in a **specific dimension** only - GetLength(0) returns the number of rows, and GetLength(1) returns the number of columns, for example. GetLength() is generally more useful when working with multidimensional arrays because it lets you reason about rows and columns separately, which is exactly what's needed when looping through the array with nested loops.

Q4. What is the difference between Array.Copy() and Array.ConstrainedCopy()?

Both methods copy a range of elements from one array to another, but **Array.ConstrainedCopy()** provides a stronger guarantee: it offers **atomic, exception-safe** behavior. If the copy operation would fail partway through (for example, due to a type mismatch), ConstrainedCopy() guarantees that **no changes are made at all** - it either fully succeeds or leaves the destination array completely untouched. **Array.Copy()** offers no such guarantee; if an error occurs partway through the copy, the destination array may be left in a partially modified state. ConstrainedCopy() is intended for use in reliability-critical code where this all-or-nothing behavior matters.

Q5. Why is foreach preferred for read-only operations on arrays?

foreach is preferred for read-only iteration because it is more concise and readable than manually managing an index variable, and it removes an entire class of bugs related to off-by-one errors or incorrect index bounds. It also communicates intent clearly: foreach signals that the code is simply reading through every element without needing to modify the collection or track position. The trade-off is that foreach does not allow you to modify the elements of the array directly through the loop variable (and does not let you easily know the current index), which is exactly why it's best suited to read-only scenarios rather than cases where elements need to be changed in place.

Q6. Why is input validation important when working with user inputs?

Input validation is important because user input can never be fully trusted or predicted - users may enter empty text, letters instead of numbers, out-of-range values, or unexpected formats, whether by mistake or intentionally. Without validation, this can cause the program to crash with an unhandled exception (such as a `FormatException` from a failed `Parse`), produce incorrect results, or in more serious cases open the door to security vulnerabilities. Validating input - for example using `int.TryParse` combined with range checks in a loop - allows the program to catch bad input gracefully, guide the user toward a correct value, and keep running reliably instead of failing unexpectedly.

Q7. How can you format the output of a 2D array for better readability?

A common approach is to loop through the array with nested loops (one for rows, one for columns) and print each element with consistent spacing, using something like `ToString().PadLeft(width)` or a format string (e.g., `string.Format("{0,4}", value)`) so that every column lines up regardless of how many digits each number has. Adding a newline after each row (rather than after each individual element) produces a clean grid layout that visually resembles rows and columns, which is much easier to read than printing every value on a single line or with inconsistent spacing.

Q8. When should you prefer a switch statement over if-else?

A switch statement is generally preferable when you're checking a **single variable against many discrete, known values** - such as a month number, a menu choice, or an enum - because it tends to be more readable than a long chain of else-if comparisons, and the compiler can sometimes optimize it more efficiently (for example, using a jump table for certain cases) than a sequential chain of comparisons. if-else remains the better choice when the conditions involve ranges, multiple different variables, or complex boolean logic that doesn't reduce to simple equality checks against one value.

Q9. What is the time complexity of Array.Sort()?

`Array.Sort()` in .NET uses an introspective sort (introsort) algorithm for most cases, which is a hybrid of quicksort, heapsort, and insertion sort. Its **average and typical-case time complexity is $O(n \log n)$** . Because it switches to heapsort if quicksort's recursion depth gets too large, it also guarantees a **worst-case time complexity of $O(n \log n)$** rather than quicksort's usual $O(n^2)$ worst case. For very small arrays, it switches to insertion sort internally for better real-world performance on small input sizes.

Q10. Which loop (for or foreach) is more efficient for calculating the sum of an array, and why?

For arrays specifically, a **for loop** is generally slightly more efficient (or at least never worse) than **foreach**, because a for loop accesses elements directly by index with no additional overhead, while **foreach** relies on an enumerator (`IEnumerator`) under the hood to step through the collection. In practice, for arrays, the C# compiler and JIT are able to optimize simple **foreach** loops into something very close to an indexed for loop, so the real-world performance difference is usually negligible. Still, when performance is absolutely critical (e.g., inside tight, high-frequency loops processing large arrays), a plain for loop remains the safer, more predictable choice.

Part02 - Q3. What happens if the user enters a value outside the range of 1 to 7 for the `DayOfWeek` enum?

`Enum.Parse()` will **not throw an exception** just because a numeric value falls outside the range of defined enum members - unlike parsing a named string like "Funday", which would throw. Because the underlying type of an enum is just an integer, `Enum.Parse()` happily converts any valid integer string (e.g., "9" or "-3") into an enum value of that type, even if no member of the enum actually corresponds to that number. The result is a `Day` value that doesn't match any of the named constants (Monday through Sunday), and printing it will simply show the raw number instead of a day name. This is why it's important to explicitly validate the result afterward using **`Enum.IsDefined()`** (or check the input range before parsing) to catch and reject out-of-range values, rather than assuming `Enum.Parse` alone will guard against invalid input.